Project Proposal

Project Title: Smart Workforce & Task Allocation System

1. Introduction

In an organization, many tasks need to be completed by a limited number of employees. Assigning these tasks manually can be difficult because employees may have different skills, workloads, availability, and deadlines.

For example, if a company has 10 tasks and 5 employees, giving two tasks to each employee does not necessarily mean the workload is equal. One employee may receive two tasks requiring 10 hours, while another may receive two tasks requiring only 3 hours.

The Smart Workforce & Task Allocation System is a web-based database system designed to solve this problem. It will manage employees, skills, projects, tasks, workload, assignments, and work history in one centralized system.

The system will recommend suitable employees for tasks by considering skill match, workload, availability, estimated working hours, priority, and deadline.

The main focus of this project is to demonstrate how a well-designed relational database and DBMS techniques can be used to solve a practical workforce management problem automatically.

2. Problem Statement

Traditional task allocation often depends on manual decisions or simple task counting. This can create several problems:

- Unequal distribution of workload.

- Difficulty finding employees with the required skills.

- Assigning tasks to already busy employees.

- Missed or delayed deadlines.

- Difficulty tracking task progress.

- Lack of assignment and work history.

- Scattered employee and task information.

- Difficulty preparing workload and performance reports.

For example:

Employee A → 2 tasks → 10 hours

Employee B → 2 tasks → 3 hours

Employee C → 2 tasks → 8 hours

Employee D → 2 tasks → 5 hours

Employee E → 2 tasks → 12 hours

Although every employee has two tasks, the workload is clearly different.

Therefore, the proposed system will use working hours and employee capacity, rather than only task count, to support better task allocation.

3. Objectives

Main Objective

To develop automation database-driven system that manages workforce information and supports skill-based, workload-aware task allocation.

Specific Objectives

The system aims to:

- Maintain employee and department information.

- Manage employee skills and skill levels.

- Manage projects and tasks.

- Store required skills for each task.

- Track employee availability and workload.

- Recommend suitable employees for tasks.

- Assign and reassign tasks.

- Track task progress and deadlines.

- Record estimated and actual working hours.

- Maintain task assignment history.

- Generate useful workforce reports.

- Demonstrate practical DBMS concepts such as normalization, SQL, relationships, constraints, views, and transactions.

4. Proposed Solution

The proposed system will provide separate interfaces for Admin, Employee.

Basic Workflow

Admin Creates Task

↓

Defines Required Skills

↓

System Finds Eligible Employees

↓

Checks Skills + Workload + Availability

↓

Calculates Recommendation Score

↓

Shows Suitable Employees

↓

System Confirm Assignment and Assign Employee

↓

Employee Works on Task

↓

Progress & Work Hours Updated

↓

Task Completed

↓

History & Reports Update

Smart Allocation Logic

The system will first identify employees who have the required skills. It will then consider their current workload, availability, and deadline suitability.

A simple scoring model will be:

Skill Match → 40%

Workload Balance → 25%

Availability → 20%

Deadline Fit → 15%

The weights can be adjusted during development and testing.

For example, if a task requires Photoshop, Illustrator, and Packaging Design, the system will find employees with those skills and then compare their current workload and availability then assign the task.

5. Key Features with ERD

5.1 Employee & Department Management

The system will store employee information such as:

- Employee ID
- Name
- Email
- Department
- Position
- Working capacity
- Availability

Employees will be connected to departments.

5.2 Skill Management

The system will maintain a list of organizational skills.

Photoshop

Illustrator

Graphic Design

UI/UX Design

Programming

Video Editing

Research

An employee can have multiple skills, and a skill can belong to multiple employees. Skill level can also be stored.

Example:

Employee A

├── Photoshop → Advanced

├── Illustrator → Intermediate

└── Graphic Design → Advanced

5.3 Project & Task Management

Admin can create projects and add multiple tasks under each project.

Each task can contain:

- Title

- Description

- Project

- Priority

- Estimated hours

- Deadline

- Status

- Required skills

Task status can follow:

PENDING → ASSIGNED → IN PROGRESS → REVIEW → COMPLETED

Additional statuses can include OVERDUE and CANCELLED.

5.4 Smart Task Allocation

The system will recommend and assign task to employees based on:

- Required skills.
- Skill level.
- Current workload.
- Availability.
- Estimated task duration.
- Priority.
- Deadline.

This is the main feature that makes the project different from a basic task-management system.

5.5 Workload Management

The system will calculate workload using estimated working hours.

For example:

Daily Capacity = 8 hours

Assigned Work = 6 hours

Workload = 6 / 8 × 100 = 75%

System can therefore identify employees who are overloaded or have available capacity.

5.6 Work Logs & Task History

Employees can record actual working hours.

The system will maintain history for:

- Task creation.
- Assignment.
- Reassignment.
- Status changes.
- Work logs.
- Completion.

This will provide an audit trail for the system.

5.7 Dashboard & Reports

The dashboard can show:

Total Employees

Active Tasks

Completed Tasks

Overdue Tasks

Average Workload

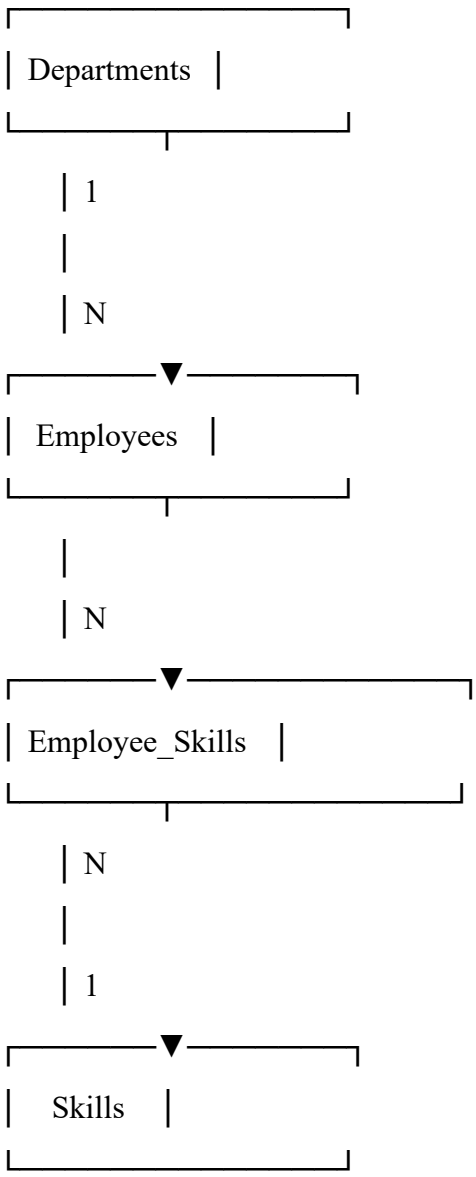
Overloaded Employees

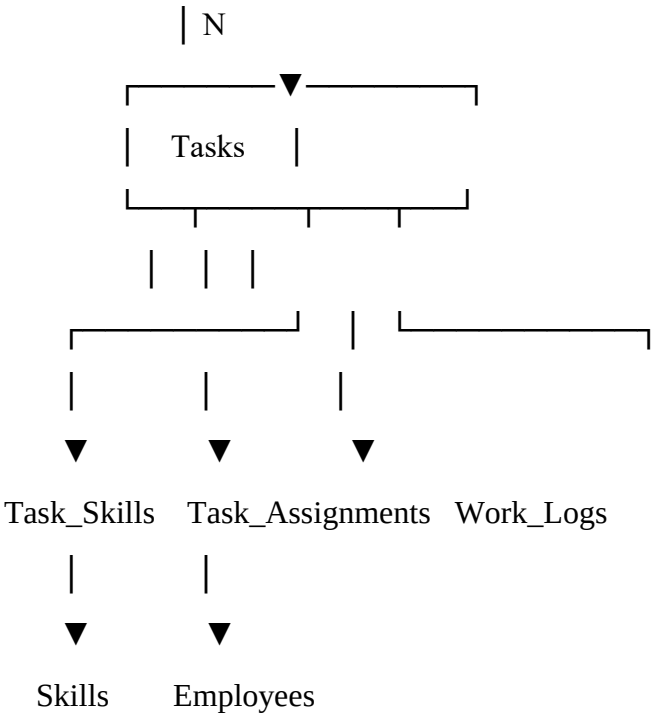
Reports can include:

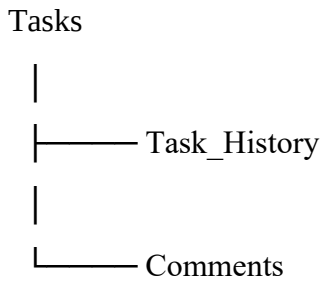
- Employee workload report.
- Employee skill report.
- Task status report.
- Overdue task report.
- Department workload report.
- Work-hour report.
- Assignment history.

5.8 ERD

The main database entities and relationships will be:







Main Tables

users

employees

departments

skills

employee_skills

projects

tasks

task_skills

task_assignments

work_logs

task_history

comments

The database will use primary keys, foreign keys, constraints, normalization, and referential integrity to maintain reliable data.

6. Technology Stack

| Layer | Technology | Purpose |

|---|---|---|

| Frontend | React.js | Web interface |

| UI Design | Tailwind CSS | Responsive and clean design |

| Backend | Node.js + Express.js | APIs and business logic |

| Database | PostgreSQL | Main relational database |

| Database Access | SQL + Prisma | Database operations |

| Authentication | JWT | Login and role-based access |

| API Testing | Postman | API testing |

| Version Control | Git + GitHub | Code management |

System Architecture

User

↓

React.js + Tailwind CSS

↓

REST API

↓

Node.js + Express.js

↓

Task Allocation Logic

↓

SQL / Prisma

↓

PostgreSQL

Because this is a DBMS-focused project, PostgreSQL and SQL will remain the core of the system. Important database operations will be demonstrated using actual SQL rather than depending completely on the ORM.

7. System Design and Development Plan

The project will follow a step-by-step development process.

Phase 1: Requirement Analysis

Identify:

- Users and roles.
- System requirements.
- Task allocation rules.
- Required reports.
- Database requirements.

Phase 2: Database Design

Prepare:

- ER Diagram.
- Entities and attributes.
- Relationships.
- Primary and foreign keys.
- Relational schema.

Phase 3: Normalization

Normalize the database up to3NF to reduce unnecessary data duplication and improve data consistency.

Phase 4: Database Development

Implement PostgreSQL tables with:

- Primary keys.
- Foreign keys.
- Constraints.
- Indexes.
- Views.
- Triggers.
- Functions.
- Sample data.

Phase 5: Backend Development

Develop the Node.js/Express backend and APIs for:

Authentication

Employees

Skills

Projects

Tasks

Assignments

Work Logs

Reports

Phase 6: Frontend Development

Develop React pages such as:

Login

Dashboard

Employees

Skills

Projects

Tasks

Task Assignment

Workload

Reports

Phase 7: Allocation Logic

Implement the task recommendation process:

Task

↓

Required Skills

↓

Eligible Employees

↓

Workload Check

↓

Availability Check

↓

Deadline Check

↓

Score Calculation

↓

Assign Task

Phase 8: Testing

The system will be tested at different levels:

- Database testing.
- SQL query testing.
- API testing using Postman.
- Frontend testing.
- Task allocation testing.
- Complete system testing.

Phase 9: Final Documentation

The final documentation will include:

- ERD.
- Database schema.
- Normalization.
- System architecture.
- Screenshots.
- Test cases.
- Results.
- User guide.

8. Expected Outcome and Future Scope

Expected Outcome

At the end of the project, the system is expected to provide a working platform where organizations can:

- Manage employees and departments.
- Manage skills and skill levels.
- Create projects and tasks.
- Identify suitable employees for tasks.
- Monitor employee workload.
- Assign and reassign tasks.
- Track task progress.

- Record working hours.
- Maintain assignment history.
- Identify overdue tasks.
- Generate useful reports.

The project will demonstrate how database design and SQL can be applied to a real-world workforce management problem*.

Future Scope

After completing the core system, the following features can be added:

AI-Based Task Classification

The system could read a task description and suggest its category, required skills, and estimated duration.

Deadline Risk Detection

The system could identify tasks that may not be completed before their deadlines.

Notifications

Email, SMS, or in-app notifications could be added for assignments and approaching deadlines.

#Mobile Application

Employees could manage tasks and work logs through a mobile application.

Advanced Analytics

Future versions could provide workload trends, estimated-vs-actual working hours, and department-level analytics.

Workload Forecasting

Historical task data could be used to estimate future workload and resource requirements.

Conclusion

The Smart Workforce & Task Allocation System combines a practical workforce-management problem with important DBMS concepts.

Its key idea is simple: do not distribute tasks only by counting them; distribute them by considering skills, workload, availability, and deadlines.

The proposed system will use:

React.js + Tailwind CSS → Node.js + Express.js → SQL/Prisma → PostgreSQL

With its relational database, ERD, smart allocation logic, workload tracking, and reporting features, the project will demonstrate both practical web development and strong database management skills.